DSR Portal Webstack Progress Report

Architecture, implementation status, cleanup summary, and next milestones

Field	Value
Project	updated-webstack-of-dsr--portal
Prepared on	11 June 2026
Scope	Full webstack: Next.js web app, Express API, Prisma/PostgreSQL schema, and legacy DSR portal frontend
Report status	Working progress report based on current local repository state

Executive Summary

The DSR Portal repository is a modernized monorepo around a Next.js web package, an Express API package, a Prisma/PostgreSQL persistence layer, and a large legacy browser portal that remains the main feature surface for DSR data entry, annexures, PDF workflows, dashboard tracking, roles, and project phases.

Current direction: Keep the working project stable while cleaning visible background effects, redundant temporary files, and low-risk noise. Large CSS override cleanup should be handled in reviewed batches because many duplicate-looking rules are cross-file fallbacks.

Webstack Overview

Layer	Current implementation	Purpose
Workspace	npm workspaces: apps/* and packages/*	Keeps API and web packages under one project root.
Web	Next.js 15, React 19, TypeScript, Tailwind-related utilities	Modern app shell and local dev server on port 3000.
Legacy UI	Static HTML/CSS/JS bundle under apps/web/public/legacy	Current DSR portal screens, dashboard, annexures, PDF preview, auth UI, and theme system.
API	Express 4, TypeScript, helmet, cors, rate limits, cookie-parser, JWT	Authenticated backend routes for projects, files, reports, users, dashboard, and PDF actions.
Database	Prisma 6 with PostgreSQL datasource	Users, projects, reports, workflow history, audit logs, and uploaded DSR files.
Jobs/files	BullMQ, multer, AWS S3 SDK	Background job and file upload/storage foundation.

Repository Metrics

Metric	Value	Notes
Tracked source files counted	153	Excludes node_modules, .git, package-lock.json, and large legacy projects.json demo data.
Approximate source lines	39k-43k	Range depends on whether generated HTML and binary-like assets are counted.
Main legacy JS files	42	Annexures, project state, PDF preview, navigation, roles, auth, theme, and dashboard behavior.
Main legacy HTML templates/files	46	Build script composes templates into login.html, home.html, and index.html.
Main CSS files	3	styles.css, premium-theme.css, and supporting style assets.

Implemented Functional Areas

- Role-aware login and portal access paths for staff, government authority, and SDLC style entry points.

- Project dashboard with district filtering, project cards, progress/status indicators, and shortcut search.

- Project lifecycle storage including active project state, phase metadata, phase locks, and child phase creation.

- Front matter, chapters, plates, annexures, more-annexures, tables, graphs, signatures, workflow, and audit views.

- PDF upload, preview, generated final PDF download, and annexure-specific PDF generation flows.

- Backend authentication, dashboard stats, project CRUD/state persistence, report workflow, file upload, jobs, users, and PDF endpoints.

- Dark/light theme handling with visual branding, watermark, and legacy app styling continuity.

Recent Progress Completed

Area	Progress	Project impact
Background system	Removed old DOM-based live-lines overlay/classes and replaced with CSS-only global grid plus four floating directional lines.	Lower DOM noise; same visible intent; no feature logic changed.
Dark mode visible lines	Removed the earlier harsh neon/drop-shadow line traces from the old system.	Dark mode background is calmer and less intrusive.
Templates/generated HTML	Regenerated login.html, home.html, and index.html from templates with asset version grid-live-lines-cleanup-20260611.	Browser cache receives the corrected CSS/JS version.
Cleanup	Deleted fix.js, an unused one-off encoding repair script with no app references.	Safe removal; not part of runtime.
Console noise	Removed harmless informational console logs from frontmatter/main paths while keeping user-facing fallbacks.	Cleaner dev console without changing UI behavior.
CSS safety	Restored malformed reset selectors and converted an orphan heading into a comment.	CSS parses cleanly and avoids accidental invalid selector behavior.

Current Verification Status

Check	Result
CSS parse check	styles.css OK; premium-theme.css OK.
Old line system search	No live-lines-overlay, interactive-grid-bg, live-line, old scan animation, dense/neon line variant, or old drop-shadow trace found.
Scoped diff for cleanup	11 legacy files changed in the cleanup scope: 179 insertions and 2,338 deletions, mostly comments/blank lines plus old line system removal.
Risk posture	No aggressive CSS override deletion performed beyond low-risk exact/unused items.

Backend/Data Model Progress

Component	Status
Prisma enums	Role, ProjectStatus, and ReportStatus are present.
Core models	User, Project, Report, WorkflowHistory, AuditLog, and DsrFile support the portal workflow.
Phase model additions	Project includes phaseNo, parentPhaseId, phaseLocked, phaseOrigin, and parent/child phase relations.
Main API routes	Auth, dashboard, files, jobs, projects, reports, users, and PDF endpoints are wired in server.ts.
Security middleware	Helmet, CORS, cookie parser, JSON body limits, route-level auth, audit mutations, and rate limits are present.

Cleanup Candidates and Risk

A CSS audit found many candidates that look overwritten or duplicated, but not all are safe to remove immediately. The legacy app loads styles differently across login, generated portal pages, and home page, so a rule that appears redundant in one page can still act as fallback in another.

Candidate	Observed count	Recommendation
Exact duplicate CSS candidates	About 52 after repair audit	Remove only when selector/file/page context is identical.
Overwritten CSS candidates	About 561	Batch by component: auth, topbar, sidebar, dashboard, annexures, PDF preview.
Important declarations	About 1,876	Do not bulk-remove; many are legacy override contracts.
Large demo/project data	projects.json excluded from LOC count	Review separately before deleting because it may preserve demo state.

Recommended Next Milestones

1. Create a visual regression checklist for login, home, dashboard, projects, workflow, front matter, annexures, PDF preview, and dark mode.

2. Split CSS cleanup into small batches and verify each batch in browser before continuing.

3. Move generated HTML handling behind the existing build script discipline so templates remain the source of truth.

4. Add API smoke tests for auth, projects, project state save/load, phase creation, and PDF endpoints.

5. Document the role/permission matrix so future UI cleanup does not accidentally expose restricted actions.

6. Decide whether legacy UI remains the primary UI or is gradually migrated into Next.js routes.

Conclusion

The webstack is functional and broad: it already covers the key DSR portal workflows from authentication and project creation through annexure editing, PDF handling, workflow tracking, and backend persistence. The safest improvement path is incremental: keep runtime behavior stable, remove unused temporary artifacts, and treat CSS override cleanup as a measured refactor with visual checks.